



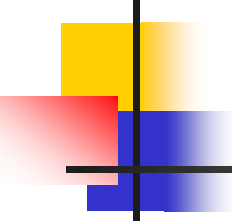
Symfony



3.1 Novedades de las últimas versiones de PHP



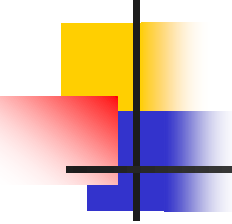
Profesor: Alejandro Amat Reina



El pseudotipo iterable

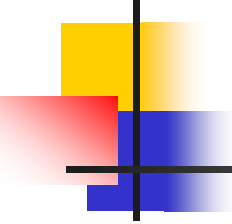
- Introducido en PHP 7.1
- Se ha introducido un nuevo pseudotipo (similar a callable) llamado iterable.
- Se puede utilizar en parámetros y tipos de retorno
- Acepta tanto arrays como objetos que implementen la interfaz Traversable (Interfaz para detectar si una clase puede recorrerse mediante foreach).

```
function iterator(iterable $iter)
{
    foreach ($iter as $val) {
        //
    }
}
```



Desestructuración de arrays simétricos

- Añadido en PHP 7.1
- La sintaxis abreviada de array (`[]`) se puede usar ahora para desestructurar arrays para asignaciones (incluyendo dentro de `foreach`) como alternativa a la sintaxis `list()` existente, la cual aún se admite.
- Desestructurar es convertir sus elementos en variables.
- Ver ayuda de la función `list` de PHP:
 - <https://www.php.net/manual/es/function.list.php>



Ejemplo

```
$datos = [  
    [1, 'Tom'],  
    [2, 'Fred'],  
];
```

```
// Estilo de list()
```

```
list($id1, $nombre1) = $datos[0];
```

```
// Estilo de []
```

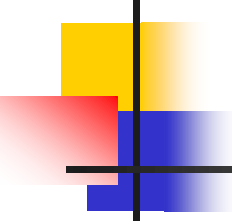
```
[$id1, $nombre] = $datos[0];
```

```
// Estilo de list()
```

```
foreach ($datos as list($id, $nombre)) {  
    // aquí va la lógica con $id y $nombre  
}
```

```
// Estilo de []
```

```
foreach ($datos as [$id, $nombre]) {  
    // aquí va la lógica con $id y $nombre  
}
```



Soporte para claves en list()

- Añadido en la versión 7.1
- Ahora se pueden especificar claves en list(), o en su nueva sintaxis abreviada [].
- Esto habilita la desestructuración de arrays con claves no enteras o no secuenciales.

```
$datos = [  
  ["id" => 1, "nombre" => 'Tom'],  
  ["id" => 2, "nombre" => 'Fred'],  
];
```

```
// Estilo de list()
```

```
list("id" => $id1, "nombre" => $nombre1) = $datos[0];
```

```
// Estilo de []
```

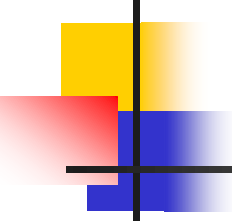
```
["id" => $id1, "nombre" => $nombre1] = $datos[0];
```

```
// Estilo de list()
```

```
foreach ($datos as list("id" => $id, "nombre" => $nombre)) {  
  // aquí va la lógica con $id y $nombre  
}
```

```
// Estilo de []
```

```
foreach ($datos as ["id" => $id, "nombre" => $nombre]) {  
  // aquí va la lógica con $id y $nombre  
}
```

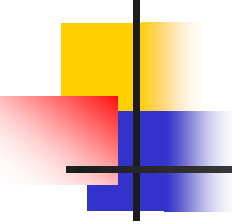


Offsets negativos en strings

- Añadido en PHP 7.1
- Se permiten offsets negativos en el manejo de strings, tanto en las funciones como en la indexación de strings

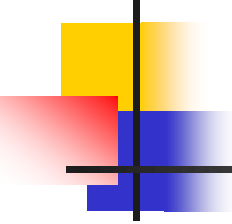
```
var_dump("abcdef"[-2]);  
var_dump(strpos("aabbcc", "b", -3));
```

- Resultado:
 - string (1) "e"
 - int(3)



nullable

- Añadido en PHP 7.1
- Las declaraciones de tipos para parámetros y valores de retorno ahora se pueden marcar como «nullable».
- Se indica poniendo un signo de cierre de interrogación delante del nombre del tipo.
- Si lo ponemos delante del tipo de un parámetro, indica que se puede pasar null como parámetro.
- Si lo ponemos delante del tipo de retorno de la función, indica que la función puede devolver null.



Ejemplo

```
function testReturn(): ?string
{
    return 'elePHPant';
}
```

```
var_dump(testReturn());
```

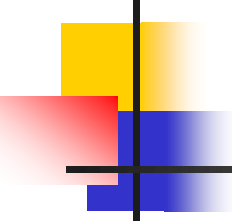
```
function testReturn(): ?string
{
    return null;
}
```

```
var_dump(testReturn());
```

```
function test(?string $name)
{
    var_dump($name);
}
```

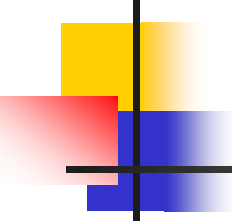
```
test('elePHPant');
test(null);
test();
```

Resultado:



Funciones void

- Añadido en PHP 7.1
- Se ha introducido un tipo de retorno, void.
- Las funciones declaradas con 'void' como su tipo de retorno deben o bien omitir su sentencia 'return' totalmente, o utilizar una sentencia 'return' vacía.
- null no es un valor de retorno válido para una función 'void'.



Ejemplo

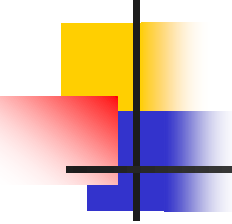
```
function intercambiar(int &$izquierdo, int &$derecho) : void
{
    if ($izquierdo === $derecho) {
        return;
    }

    $tmp = $izquierdo;
    $izquierdo = $derecho;
    $derecho = $tmp;
}
```

```
$a = 1;
$b = 2;
var_dump(intercambiar($a, $b), $a, $b);
```

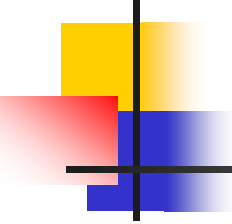
Resultado:

```
null
int(2)
int(1)
```



Nuevas funciones

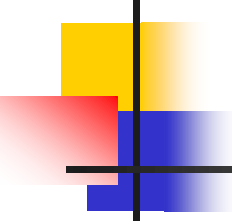
- Añadido en PHP 7.3
- Se añaden dos nuevas funciones de arrays:
 - `array_key_first()`: Obtiene la primera clave de un array
 - `array_key_last()`: Obtiene la última clave de un array



Operador de asignación de fusión nula ??=

- Introducido en PHP 7.4
- Este operador nos permite asignar un valor por defecto solo en el caso de que una variable no esté inicializada
- No confundir con el operador de fusión de null ??

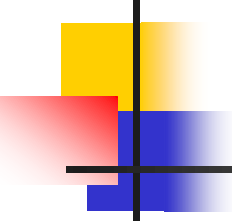
```
$array['key'] ??= computeDefault();  
// is roughly equivalent to  
if (!isset($array['key'])) {  
    $array['key'] = computeDefault();  
}
```



Desempaquetar un array en otro array

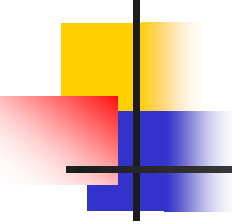
- Introducido en PHP 7.4
- Se pueden fusionar dos arrays introduciendo uno dentro del otro
- La sintaxis es la siguiente:

```
$parts = ['apple', 'pear'];  
$fruits = ['banana', 'orange', ...$parts, 'watermelon'];  
// ['banana', 'orange', 'apple', 'pear', 'watermelon'];
```



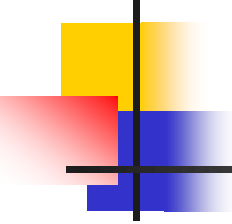
strip_tags admite un array de nombres de etiqueta

- Introducido en PHP 7.4
- strip_tags () ahora también acepta un array de etiquetas con las etiquetas que se permiten
- Anteriormente hacíamos esto:
 - `strip_tags ($ str, '<a> <p>');`
- Ahora podemos hacer esto:
 - `strip_tags ($ str, ['a', 'p]);`



Funciones de flecha (Arrow functions)

- Se introducen en PHP 7.4
- Sintaxis más concisa para las funciones anónimas.
- Tienen la forma básica `fn (argument_list) => expr.`
- Soportan las mismas características que las funciones anónimas, excepto que el uso de variables del ámbito padre siempre es automático.
- Cuando una variable utilizada en la expresión se define en el ámbito padre será implícitamente capturada por valor.



Ejemplo: captura de variables por valor automáticamente

```
$y = 1;
```

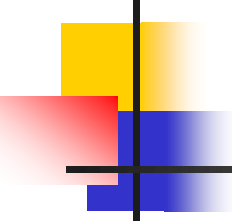
```
$fn1 = fn($x) => $x + $y;
```

```
// equivalent to using $y by value:
```

```
$fn2 = function ($x) use ($y) {  
    return $x + $y;  
};
```

```
var_export($fn1(3));
```

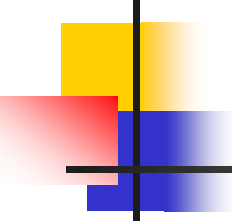
Resultado: 4



Ejemplo: Lo mismo, pero con funciones anidadas

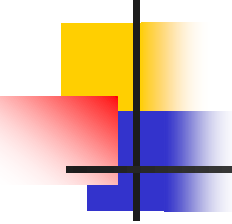
```
$z = 1;  
$fn = fn($x) => fn($y) => $x * $y + $z;  
// Outputs 51  
var_export($fn(5)(10));
```

Resultado: 51



Características de la funciones arrow

- La sintaxis de las funciones arrow permite:
 - tipos de parámetros y de retorno
 - valores por defecto
 - parámetros por valor, así como por referencia al pasar y al regresar.



Ejemplos válidos

fn(**array** \$x) => \$x;

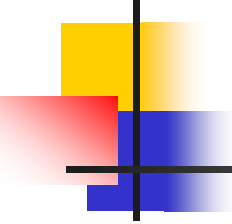
static fn(): *int* => \$x;

fn(\$x = 42) => \$x;

fn(&\$x) => \$x;

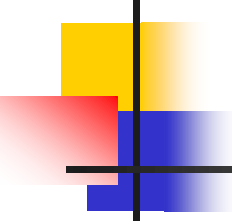
fn&(\$x) => \$x;

fn(\$x, ...\$rest) => \$rest;



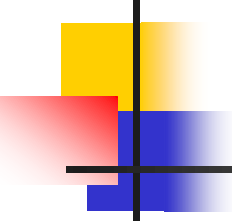
Uso de variables por valor

- Cuando usamos una variable del ámbito superior en una función de flecha, lo hacemos por valor.
- Esto es equivalente a realizar un `use($x)` para cada variable `$x` usada dentro de la función de la flecha, pero no podremos modificar esta variable.
- Si necesitamos modificar las variables externas usaremos funciones anónimas en su lugar.



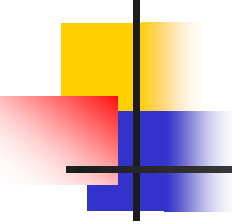
Ejemplo

```
$x = 1;  
$fn = fn() => $x++; // No tiene ningún efecto  
$fn();  
var_export($x); // Salida 1
```



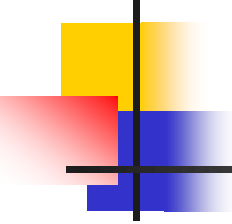
Nuevas funciones de string

- Introducido en PHP 8.0
- `str_contains`: Determina si un string contiene un substring
`str_contains(string $haystack, string $needle): bool`
- `str_ends_with`: Determina si un string termina con un substring
`str_ends_with(string $haystack, string $needle): bool`
- `str_starts_with`: Determina si un string empieza con un substring
`str_starts_with(string $haystack, string $needle): bool`



Parámetros nombrados

- Introducida en PHP 8.0
- Ofrece la posibilidad de indicar el nombre de un parámetro, de manera que ya no será necesario pasar los parámetros en el orden establecido en la cabecera de la función.
- En la cabecera de la función habrá que indicar el nombre del parámetro seguido del carácter ':' y después el nombre de la variable que almacenará el parámetro.
- Se pueden mezclar los parámetros nombrados con los parámetros posicionales, pero en este caso los nombrados tienen que estar al final de la lista de parámetros.



Ejemplo

```
myFunction(paramName: $value);  
array_foobar(array: $value);
```

```
// NOT supported.
```

```
function_name($variableStoringParamName: $value);
```

```
// Using positional arguments:
```

```
array_fill(0, 100, 50);
```

```
// Using named arguments:
```

```
array_fill(start_index: 0, count: 100, value: 50);
```

```
array_fill(value: 50, num: 100, start_index: 0);
```

```
htmlspecialchars($string, double_encode: false);
```

```
// Same as
```

```
htmlspecialchars($string, ENT_COMPAT | ENT_HTML401, 'UTF-8', false);
```

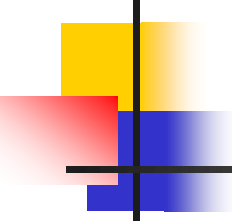
```
function foo($param) { ... }
```

```
foo(param: 1, param: 2);
```

```
// Error: Named parameter $param overwrites previous argument
```

```
foo(1, param: 2);
```

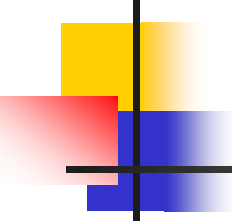
```
// Error: Named parameter $param overwrites previous argument
```

Union types

- Introducido en PHP 8.0
- Podemos declarar más de un tipo para parámetros de función y/o método, tipos de retorno y propiedades de clase.
- Además, también podemos declarar el tipo null para aquellos tipos que admitan valores nulos

```
class Example {  
    private int|float $foo;  
    public function squareAndAdd(float|int|null $bar): int|float {  
        $bar = $bar ?? 0;  
        return $bar ** 2 + $this->foo;  
    }  
}
```



Errores en tipos de union

- Se producirá un error en las declaraciones de tipo si se utilizan tipos redundantes
- Por ejemplo `int|string|INT` producirá un error
- `object | nombreClase` producirá un error

function foo(): int|**INT** {} *// Disallowed*

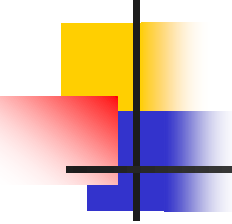
function foo(): bool|**false** {} *// Disallowed se admite el tipo false por compatibilidad con funciones antiguas, pero no se admite el tipo true*

use A **as** B;

function foo(): A|**B** {} *// Disallowed ("use" is part of name resolution)*

class_alias('X', 'Y');

function foo(): X|**Y** {} *// Allowed (redundancy is only known at runtime)*

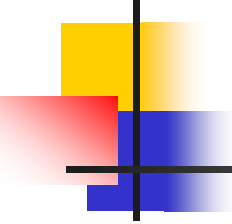


Expresión match

- Introducido en PHP 8.0
- Similar a switch, pero devolviendo un valor.
- A diferencia de switch, la comparación es una verificación de identidad (===) en lugar de una verificación de igualdad débil (==).

```
$return_value = match (subject_expression) {  
    single_conditional_expression => return_expression,  
    conditional_expression1, conditional_expression2 => return_expression,  
};
```

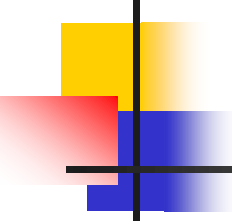
- El resultado de una expresión match no tiene porque ser usado
- La expresión debe terminar con ;



Diferencias con switch

- Como en el switch, las expresiones match se ejecutan condición a condición
- Las expresiones condicionales solo se evalúan si todas las expresiones condicionales anteriores no coincidieron con el valor del match.
- Solo se evaluará la expresión de retorno correspondiente a la expresión condicional coincidente.

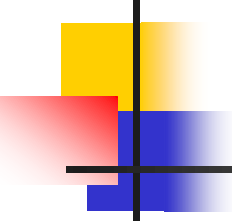
```
$result = match ($x) {  
    foo() => ...,  
    $this->bar() => ..., // $this->bar() isn't called if foo() === $x  
    $this->baz => beep(), // beep() isn't called unless $x === $this->baz  
    // etc.  
};
```



Más de un valor de comparación

- Las expresiones de un match pueden contener más de un valor separados por comas
- Se interpretarán como una expresión or

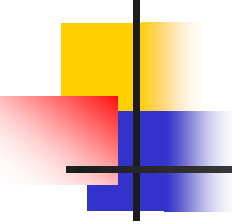
```
$result = match ($x) {  
    // This match arm:  
    $a, $b, $c => 5,  
    // Is equivalent to these three match arms:  
    $a => 5,  
    $b => 5,  
    $c => 5,  
};
```



Valor por defecto

- Al igual que en switch podemos incluir una expresión default que será la que se use en el caso de que las demás expresiones no hayan coincidido.

```
$expressionResult = match ($condition) {  
    1, 2 => foo(),  
    3, 4 => bar(),  
    default => baz(),  
};
```

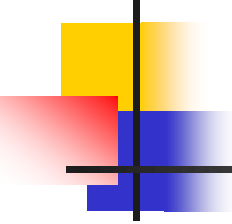


Errores en match

- La expresión match debe ser exhaustiva, es decir, siempre debe haber una expresión que coincida con el valor buscado, de no ser así se producirá un error.

```
$condition = 5;
```

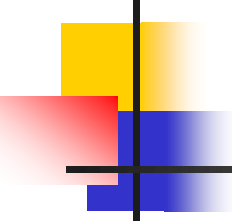
```
try {  
  match ($condition) {  
    1, 2 => foo(),  
    3, 4 => bar(),  
  };  
} catch (\UnhandledMatchError $e) {  
  var_dump($e);  
}
```



Uso con rangos de enteros

```
$age = 23;
```

```
$result = match (true) {  
    $age >= 65 => 'senior',  
    $age >= 25 => 'adult',  
    $age >= 18 => 'young adult',  
    default => 'kid',  
};
```

Uso con contenidos de string

```
$text = 'Bienvenue chez nous';
```

```
$result = match (true) {  
  str_contains($text, 'Welcome') || str_contains($text, 'Hello') => 'en',  
  str_contains($text, 'Bienvenue') || str_contains($text, 'Bonjour') => 'fr',  
  // ...  
};
```